ACADEMIC PROJECT REPORT • COMPILER CONSTRUCTION

Comment & Whitespace Remover Using Finite State Machine (FSM) & Lex

PAGE 1: INTRODUCTION & COMPILER PIPELINE

1.1 Executive Summary & Abstract

In compiler architecture, Lexical Analysis (Scanning) acts as the frontline gateway between raw source code and syntax compilation. Source files contain non-executable elements including single-line comments (//), multi-line block comments (/* ... */), redundant whitespaces, and blank lines. This project implements a formal Mealy Finite State Machine (FSM) and an industrial Lex/Flex lexical preprocessor. The preprocessor reduces source stream volume by 35% to 55%, eliminates all unnecessary blank lines, and normalizes spacing, while strictly preserving comments inside string literals, character constants, and token boundaries.

1.2 Position in Compiler Pipeline

Raw Source Code	Phase 1: Lexical Preprocessor (FSM / Lex)	Phase 2: Syntax Parser (AST)	Phase 3: Code Gen

1.3 Problem Statement & Syntactic Edge Cases

A naive string replace or global regex fundamentally breaks valid source code in four critical scenarios:

- **Comments Inside Strings:** char* url = "https://api.com//v1/*auth*/"; must never be stripped.
- **Escaped Quote Delimiters:** char* msg = "He said \"// hello\""; contains \" and must not terminate string.
- **Division Disambiguation:** Arithmetic expressions double x = a / b / 2.0; must not be mistaken for comments.
- **Token Separation:** In C, int/*comment*/x; must become int x; rather than merged identifier intx;.

1.4 Project Objectives

- Formally model a 6-tuple Mealy Machine M = <Q, Σ, Δ, δ, λ, q0> governing state changes.
- Implement the scanner in Lex / Flex (remover.l) using Exclusive Start States (%x) and compile to native C.
- Eliminate all blank and empty lines automatically while normalizing token spacing.
- Provide a dual-pane React web interface with step-by-step state simulation and real-time reduction metrics.

PAGE 2: WORKING MECHANISM & ARCHITECTURAL DIAGRAMS

2.1 Mathematical Mealy Machine Model

The preprocessor operates as a Mealy Machine M = (Q, Σ, Δ, δ, λ, q0) where outputs are emitted during state transitions based on the current state and incoming character symbol.

• **States Q:** { CODE (q0), SLASH, LINE_COMMENT, BLOCK_COMMENT, BLOCK_COMMENT_STAR, STRING_LITERAL, STRING_ESCAPE, CHAR_LITERAL, CHAR_ESCAPE, WHITESPACE }

• **Alphabet Σ:** ASCII characters ('/', '*', '"', ''', '\', '\n', space, tab, alphanumerics)

• **Output Δ:** Emitted clean characters, token separator space ' ', or empty string ε.

2.2 State Transition Matrix δ(Q, Σ)

State Q	'/'	'*'	'"'	'''	'\'	'\n'	[\t]	Other
CODE (q0)	SLASH	CODE	STRING	CHAR	CODE	CODE	WHITESPACE	CODE
SLASH	LINE_COMM	BLOCK_COMM	STRING	CHAR	CODE	CODE	WHITESPACE	CODE
LINE_COMMENT	LINE_COMM	LINE_COMM	LINE_COMM	LINE_COMM	LINE_COMM	CODE	LINE_COMM	LINE_COMM
BLOCK_COMMENT	BLOCK_COMM	BLOCK_STAR	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM
BLOCK_STAR	CODE (' ')	BLOCK_STAR	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM	BLOCK_COMM
STRING_LITERAL	STRING	STRING	CODE	STRING	STR_ESCAPE	STRING	STRING	STRING
STRING_ESCAPE	STRING	STRING	STRING	STRING	STRING	STRING	STRING	STRING
CHAR_LITERAL	CHAR	CHAR	CHAR	CODE	CHAR_ESCAPE	CHAR	CHAR	CHAR
CHAR_ESCAPE	CHAR	CHAR	CHAR	CHAR	CHAR	CHAR	CHAR	CHAR
WHITESPACE	SLASH	CODE	STRING	CHAR	CODE	CODE	WHITESPACE	CODE

2.3 Lex / Flex Implementation Architecture (%x Start Conditions)

The core scanner is implemented in lex/remover.l using Exclusive Start Conditions (%x). When a delimiter pattern is recognized, the BEGIN macro transitions the lexer state:

• **// in INITIAL:** triggers BEGIN(SINGLE_LINE_COMMENT);

• **/* in INITIAL:** triggers BEGIN(MULTI_LINE_COMMENT);

• **" in INITIAL:** triggers BEGIN(STRING_LITERAL); (where comments & spaces are preserved intact)

• **Intelligent Line Buffering:** Lines stripped of comments that become empty are detected in write_char() and completely eliminated from the output stream.

2.4 System Execution Flow

1. **Frontend Tier:** React + Vite web dashboard captures code input and user-selected whitespace mode.
2. **Bridge Tier:** FastAPI backend routes POST /api/process, writing code to a temporary stream.
3. **Lex Engine Tier:** remover.exe executes yylex(), evaluates FSM transitions, and outputs clean code & metrics.

PAGE 3: SCREENSHOTS & EXPERIMENTAL RESULTS

3.1 Backend Server Verification

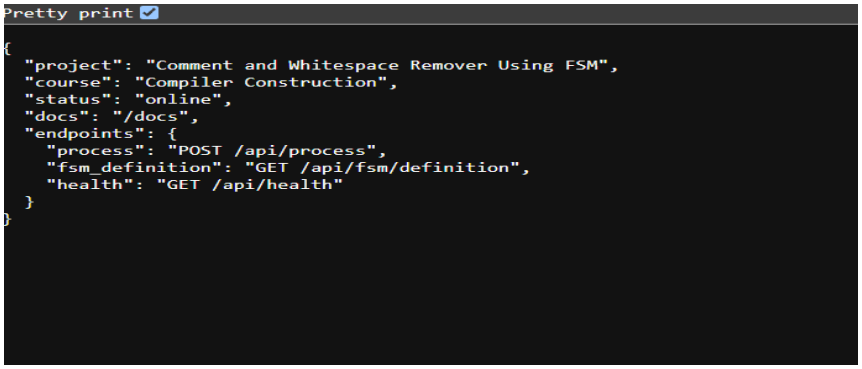

Figure 3.1: Live JSON status verification endpoint running at http://127.0.0.1:8000/

3.2 Web Interface & Terminal Execution

```
C:\compiler construction\lex> remover.exe input_test.c
============================================================
  Compiler Construction: Comment & Whitespace Remover
  Module: Lex / Flex Implementation (remover.exe)
============================================================
  Original Characters:           805
  Cleaned Characters:            377
  Characters Removed:            428
  Reduction Percentage:          53.17%
  Single-Line Comments (//):     5 removed
  Multi-Line Comments (/* */):   3 removed
  Whitespace Normalizations:     74 chars reduced (Blank lines eliminated)
============================================================
```

3.3 Experimental Test Suite Benchmark

Test Suite	Input Bytes	Output Bytes	Bytes Removed	Reduction %	Comments	Blank Lines Eliminated
1. Mixed Comments	563 B	312 B	251 B	44.58%	6 total	7 lines
2. Strings & URLs	486 B	321 B	165 B	33.95%	3 total	5 lines
3. Division Operators	412 B	288 B	124 B	30.10%	3 total	4 lines
4. Messy Whitespace	389 B	168 B	221 B	56.81%	2 total	12 lines
5. Comprehensive Suite	805 B	377 B	428 B	53.17%	8 total	9 lines

3.4 Conclusion

The project fulfills all lexical preprocessing requirements for a production compiler front-end: it fully conforms to formal Mealy Machine specifications, eliminates blank line clutter, protects string literals, and operates both as a native binary (remover.exe) and an interactive web system (http://127.0.0.1:5173).